Artificial Intelligence Lab

Dibyasundar Das

1 October 2026

LAB 09

1. **Propositional Logic Engine (Model Checking)**
   Sentence representation. Represent every propositional sentence as a nested tuple:

Logic	Python representation
P	`'P'`
$\neg P$	`('not', 'P')`
$P \wedge Q \wedge R$	`('and', 'P', 'Q', 'R')`
$P \vee Q$	`('or', 'P', 'Q')`
$P \Rightarrow Q$	`('implies', 'P', 'Q')`
$P \Leftrightarrow Q$	`('iff', 'P', 'Q')`

   - Write a function that evaluates any sentence in a given model (a dictionary of truth values).
   - Print the **truth table** of any sentence.
2. Implement **entailment by truth-table enumeration**: $KB \models \alpha$ iff α is true in every model in which KB is true.
   - Check validity and satisfiability of a sentence.
   - Test your code on:
     1. $(P \Rightarrow Q) \Leftrightarrow (\neg Q \Rightarrow \neg P)$ — print its truth table; is it valid?
     2. $KB = \{P \Rightarrow Q,\ Q \Rightarrow R,\ P\}$.
        1. Does $KB \models R$?
        2. Does $KB \models \neg R$?
     3. Is $P \vee \neg P$ valid? Is $P \wedge \neg P$ satisfiable?

> **Note**
>
> **Valid (tautology):** a sentence is valid if it is **true in every model**, meaning every possible assignment of true/false to its symbols.
>
> - Example: $P \vee \neg P$ is true whether P is T or F.
> - In code: `is_valid(s)` returns `True` only if `pl_true(s, m)` holds for **all** 2^n models.
>
> **Satisfiable:** a sentence is satisfiable if it is **true in at least one model**.

- Example: $P \wedge Q$ is true when P = T and Q = T.
- In code: `is_satisfiable(s)` returns `True` if **any** model makes `s` true.

Unsatisfiable (contradiction): true in **no** model, for example $P \wedge \neg P$.

Sentence	Valid?	Satisfiable?
$P \vee \neg P$	Yes	Yes
$P \wedge Q$	No	Yes
$P \wedge \neg P$	No	No

3. Modify the WUMPUS world problem designed in last lab to use propositional logic to find the path to gold.